once the code is being compiled then it creats few other files also with it

they are:

1. Source Code (.ino) - Code written by the programmer that include all the libraries and different functions.

2. Object Code (.0) - The compiler (e.g., avr-gcc for AVR Arduinos) takes each source code file (.ino, .cpp) and translates it into an object file (.o).

Object files contain machine code for individual source files, but they are not yet complete executables because they don't have all the references to libraries or other parts of the program resolved.

3. Linkinf to ELF (.elf) - The linker (e.g., avr-ld for AVR Arduinos) takes all the object files (.o) from your sketch and any libraries you're using, and combines them into a single, complete executable file.

This is where external function calls (like Serial.begin(), digitalWrite(), etc.) are resolved and linked to their actual implementations in the Arduino core libraries.

The direct output of the linker is typically the .elf file.

4. Conversion to HEX and BIN (.hex, .bin) - After the .elf file is generated, utility programs (like avr-objcopy for AVR Arduinos) are used to extract specific sections of the .elf file and convert them into formats suitable for flashing. The .hex and .bin files are derived from the .elf file.

Explanation of Each File :

Source Code - Written by the programmer

1) .elf (Executable and Linkable Format)

What it is: This is the primary output of the linker. It's a highly structured file format. Think of it as the "master" compiled program.

What it contains:

Executable Machine Code: The actual instructions for your microcontroller.

Data Sections: Global variables, constants.

Symbol Table: Information about all the functions and variables in your program, their names, and their memory addresses. This is extremely useful for debugging.

Debugging Information: Data that debuggers use to map machine code back to your original source code lines.

Relocation Information: Details about how different parts of the code relate to each other and where they will reside in memory.

Purpose: While it contains the full program, it's not typically used directly for flashing on Arduino (especially AVRs). Its main purpose is for debugging, advanced analysis, and as an intermediate step from which other flashable formats are derived.

2) .hex (Intel HEX Format)

- What it is: This is a text-based (ASCII) file that represents the binary machine code in a standardized format for programming microcontrollers.
- What it contains:
    - Machine Code Bytes: Your program's instructions.
    - Memory Addresses: Specifies exactly where each byte of code should be placed in the microcontroller's flash memory.
    - Record Type: Indicates if a line contains data, an end-of-file marker, or an extended address.
    - Checksums: Each line has a checksum to ensure data integrity during transfer to the microcontroller.
- Purpose: This is the most common format used to upload (flash) programs to AVR-based Arduino boards (like Uno, Nano, Mega, Pro Mini, etc.) using tools like avrdude. It's efficient for transferring data over serial lines and includes crucial addressing information.

3) .bin (Raw Binary Format)

- What it is: This is a raw, unformatted binary image of your compiled code. It's simply a direct dump of the memory contents.
- What it contains: Just the raw bytes of your program code and data, exactly as they would appear in the microcontroller's memory.
- Purpose:
    - Less common for AVR Arduinos for direct flashing, as .hex is preferred due to its addressing and checksum capabilities.
    - Very common for flashing other microcontrollers, especially ARM-based chips and popular IoT boards like ESP32 and ESP8266. These platforms often have bootloaders that understand raw .bin files and sometimes use multiple .bin files for different memory sections (e.g., bootloader, application, data partitions).
    - Useful for direct memory programming where the flashing tool already knows the target memory addresses.

4) .eep (EEPROM Image File)

- What it is: This file contains the data that is intended to be programmed into the microcontroller's EEPROM (Electrically Erasable Programmable Read-Only Memory).
- What it contains:
    - Any const variables or arrays that you explicitly place in EEPROM memory using specific compiler attributes (e.g., __attribute__((section(".eeprom"))) or using PROGMEM for EEPROM, though PROGMEM typically targets flash).
    - Data that you might have pre-defined to be loaded into EEPROM at the time of flashing, rather than writing to it dynamically from your code.
- Purpose: To initialize or store persistent data in the microcontroller's EEPROM during the flashing process. EEPROM retains its data even when power is removed, making it suitable for configuration settings, calibration data, or small amounts of user data that need to persist across reboots. On many Arduinos, if you don't explicitly put data into EEPROM in your code, this file might be empty or not generated.

Order of Generation (Simplified Serial Order)

1. **.o (Object Files)**: These are generated first, for each compiled source file.
2. **.elf (Executable and Linkable Format)**: This is generated by the linker, combining all the .o files and libraries. This is the first "complete" program file.
3. **.hex and .eep**: These are typically generated *from* the .elf file using utility programs like objcopy. The .hex file extracts the flash memory segment, and the .eep file extracts the EEPROM segment.
4. **.bin**: If generated for your board, this would also typically be generated *from* the .elf file. For AVR, the .hex is the primary "binary" output for flashing, but .bin can be created from .elf as well. For ESP32/ESP8266, .bin is the main flashable output derived from .elf.

Source Code -> Object Files (.o) -> ELF (.elf) -> HEX (.hex) / EEPROM (.eep) / BIN (.bin)